SIMATS ENGINEERING

Student name :- Sandhiya Chandran

Registration num :- 192526034l

Course name :- Database management Systems

Course Code :- CSA0503

Course faculty :- Dr. S. SANKAR

Date of Submission :- 03/11/2024

No. of Pages :- 07

ASSIGNMENT - 2

TASK - I Alternative Schema

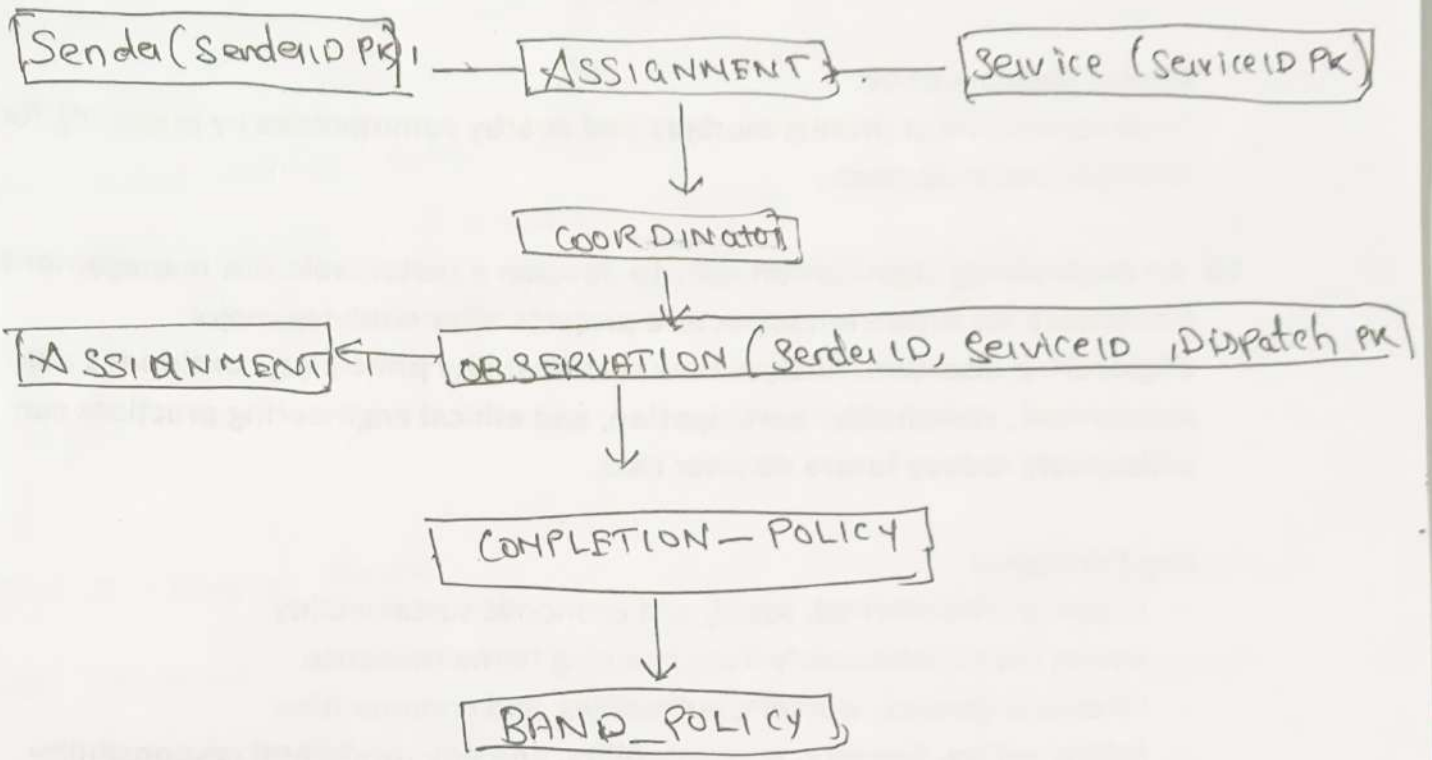
Flat :- One table containing ; Services, coordinator, observation and fields PK = (senderID, ServicesID, Dispatchwave). It repeats many

Partial :- Separate SENDER and SERVICE tables, but keep coordinat and policy details in the observation table. Some repetition remai

Normalized :- SENDER(senderID PK, sendername, Route group); Service(ServiceID PK, ServiceCode. UNIQUE, Servicename, Volume weight); COORDINATOR (CoordinatorID PK, Coordinatorname, Hub); ASSIGNMENT SenderID, ServiceID, CoordinatorID; OBSERVATION (SendID, Service COMPLETION_POLICY (completion PK, Service Band); BAND_POLICY (Service Band PK, Route Factor).

dependencies. Prime attributes (SenderID,

ER diagram :-



TASK 2 : Functional dependencies:

SenderID and ServiceID determine partial details because they are parts of the composite observation key. Coordinator and policy dependencies are transitive. the full observation key determines completion. Do not assume dependencies from sample rows alone

TASK 3 : closures, Keys and minimal cover.

The closure of (SenderID, ServiceID, Dispatch wave) contains all attributes So it is a candidate key. Since servicecode uniquely determines serviceID {SenderID, Servicecode, Dispatch wave} is another candidate key. Each key component is necessary By transitivity (SenderID, ServiceID) → coordinator Name follows from the assignment and coordinator dependencies.

dependencies implied by other dependencies. Prime attributes: ServiceID, ServiceCode. Dispatch wave.

TASK 4 – Anomalies

Insertion: a service may not be stored without an observation in a flat table. Deletion: deleting its last observation may remove the only service details. Update: changing ServiceName requires updating many copies. Partial design still repeats CoordinatorName, Hub and PolicyFlag. Count affected rows with SELECT COUNT(*) FROM SwiftSpan_Flat WHERE ServiceID=1

TASK 5 – Normal forms

1NF requires atomic values. The flat design violates 2NF because SenderID and ServiceID determine non-key details. It violates 3NF/BCNF due to transitive dependencies. In the normalized design, each determinant is a key in its own table. So the listed relations satisfy BCNF. Example 3NF but not BCNF: R(A,B,D), AB → D and D → B; D is not a superkey, but B is prime.

TASK – 6 – Lossless join, LINF and 5NF

A lossless join reconstructs exactly the original rows. Splitting sender details into SENDER is lossless because SenderID is its key. Dependency preservation means each rule can be checked without joins. Independent nodes and document types can be stored in Service_Node (ServiceID, NodeID) and Service_DOCUMENT (ServiceID, DOCUMENT TYPE, NodeID)

for 4NF. A three-way approval relation may be split into pairwise projections for 5NF only if the business rule supports it.

TASK 7 Relational algebra / calculus.

Q1: Select observations where SenderID=1 and Dispatch wave=4

The weighted-index formula is not specified in the uploaded brief

Q2: use division to find senders meeting completion ≥ 39 for every required

Q3: observed wave-4 services minus services having completion < 39

Q4: Compare condition AVG(completion) with the observation-weighted hub average.

Q5. select senders with completion < 39% in at least 3 distinct waves using COUNT(DISTINCT DispatchWave) ≥ 3. Aggregate extend classical algebra / calculus.

TASK 8 — SQL Implementation

```
CREATE TABLE Sender(
 SenderID INT PRIMARY KEY
 Sendername VARCHAR(100) NOT NULL)
 RouteGroup VARCHAR(50) NOT NULL);

CREATE TABLE Service (
ServiceID INT PRIMARY KEY,
ServiceCode VARCHAR(30) NOT NULL UNIQUE
ServiceName VARCHAR(100) NOT NULL,
```

```
CREATE TABLE coordinator (
  coordinatorID INT PRIMARY KEY,
  coordinatorname VARCHAR(100) NOT NULL,
  Hub VARCHAR(80) NOT NULL);

CREATE TABLE Assignment (
  SenderID INT NOT NULL,
  ServiceID INT NOT NULL,
  coordinatorID INT NOT NULL,
  PRIMARY KEY (SenderID, ServiceID).
  FOREIGN KEY (SenderID) REFERENES Sender(SenderID) on DELETE RESTRICT
  FOREIGN KEY (coordinatorID) REFERENCES coordinator (CoordinatorID)
                                        on DELETE RESTRICT);

CREATE TABLE observation(
  SenderID INT NOT NULL,
  ServiceID INT NOT NULL,
  Dispatchwave INT NOTNULL,
  Completion INT NOT NULL CHECK (completion Between 0 and 100),
  PRIMARY KEY (SenderID, ServiceID, Dispatchwave),
  FOREIGN KEY (SenderID, ServiceID)
  REFERCES Assignment (SenderID, ServicesID) ON DELETE RESTRICT);
```

Example query:- Observed services in wave 4 with no completion below 39%

```
SELECT DISTINT O.ServiceID
FROM observation O
where O.Dispatchwave = 4
AND NOT EXISTS (
  Select 1 FROM observation X
  where X.ServiceID=O.serviceID
  AND X.Dispatchwave=4 and X.completion<39
```

TASK 9 - Experimental results :

Test 10,000, 100,000 and 600,000 observations. Record rows, data/index bytes, median query time, integrity message and whether outputs match. Use information_schema.tables for storage and EXPLAIN for query plans. Fill in actual results from MySQL.

TASK - 10 - Theory Vs Experiment

Flat tables need more repeated updates. Normalized tables reduce anomalies but may need more joins. Actual speed depends on indexes, caching, engine, query plan and hardware. Explain measure; do not claim local tests guarantee production speed.

TASK - 11 - Final Schema recommendation.

Recommend the normalized schema as the source of truth because it reduces repeated facts and update errors. Its costs is extra joins. Support the decision with measured timings, bytes and rows changed; do not claim local tests guarantee production speed.

TASK - 12 - Re-evaluate design:

$5,000 look-ups / hour is about 1.28 look-ups / second on average. Keep the normalized source and test on indexed read projection. If faster reads are needed. An ordinary view is not materialized; a read and stores pre-joined data and needs refresh control. Prefer transactional refresh, use a version / watermark Prevent readers seeing partial refreshes, and verify the projection matches the source, compare real latency. storage and refresh cost.

Conclusion:- use the normalized schema as the source of truth, add a controlled read projection only if measurements show better performance with covered sufficiently fresh results.